1. Using this manual and your first program

This manual describes EV3 Block Lab v0.25.6. Open the corresponding block group in the editor, then read its chapter. It is a guide to this application's behavior, not a claim of complete compatibility with LEGO software.

Start with the Sound only sample to test communication without moving motors. A first driving program is: Start -> set movement motors left B / right C -> set wheel diameter 56 mm -> move forward 20 cm at 20%. Change the ports and diameter to match the actual robot. Turns also require track width.

Chapter	Contents
1	Using this manual and your first program
2	Screen, language, files, connection and transfer
3	Start and parallel execution
4	Events and messages
5	Motors
6	Movement
7	Sensors
8	Sound
9	Control
10	Operators and PID
11	Display
12	Light
13	Data: text and numeric lists
14	Recording: CSV logs
15	Variables
16	My Blocks and results
17	Parameters
18	Samples and hardware testing
19	Errors and limits
20	EV3 Classroom comparison and purpose

The application generates programs for standard EV3 firmware. Programs saved on the brick can run after the computer is disconnected and after the brick is restarted. The owner has tested basic Bluetooth transfer and operation on Windows. New features, USB and Chromebook combinations still need hardware tests. Automated tests and browser checks are not hardware verification.

2. Screen, language, files, connection and transfer

Japanese and English

Choose 日本語 or English at the top right. The selection is remembered in this browser. Buttons, block labels, menus and help change language; names and text that you enter remain unchanged. A saved project can be opened in either language. The language is a browser preference, not part of the robot's program.

Changing language rebuilds the displayed blocks from their saved data, preserving their contents and connections. Undo history is cleared. Finish any transfer or communication before switching. PDF manual and HTML manual open the selected language's edition. Sample variable names and sample text may remain Japanese because they are program data.

Editor controls

Control	Meaning
Project name	EV3 destination folder, for example Class3. Use 1-31 ASCII letters, digits, hyphens or underscores.
Program name	File name without extension, using the same character rules. LineTrace saves as LineTrace.ev3lab and transfers as LineTrace.rbf.
Save / Open	Download or open an editable project. Opening replaces the current work. Save first if needed.
New program	Begin a new project. Export reusable My Blocks first.
More: Export / Import My Blocks	Save definitions and dependencies as my-blocks.ev3blocks, or add them to the current project. Duplicate definition names must be resolved.
Sample	Replace the current program with a demonstration. Confirm replacement if edits are pending.
Undo / Redo	Undo or restore editing actions. File loads and language changes start a new history.
Show all	Close the palette and fit all blocks in the available workspace. Does not change block connections.

Select a category to open its palette. It stays open while taking blocks out. Choose another category to switch; click blank workspace to close it. Right-click Duplicate copies the selected block, its inputs, nested blocks and everything connected below it. Blocks above it are not copied.

Connect / transfer opens a panel beside the workspace and reduces the editor's width. Sample and edit controls remain available. Close or Esc restores the width; opening and closing the panel does not itself connect, disconnect or transfer. On narrow screens controls wrap and the panel scrolls separately.

Saving and backup

Autosave stores the current project in this browser for this site. It can restore after a normal shutdown, but clearing browser data, using another device or changing the site address does not carry that data across. Download a project at the end of a lesson. The GitHub source backup does not automatically contain pupils' projects.

.ev3lab is the editable project, .ev3blocks is a My Block library, and .rbf is the brick executable. Earlier files created by this application with .json or .ev3 extensions may be read. These formats are not interchangeable with LEGO's own .ev3 projects.

Connection and transfer

Action	Behavior
Bluetooth	Pair EV3 in the operating system first, then select its Web Serial port. Windows may show a COM number instead of EV3's name. Select the matching outgoing port.
USB	Choose USB, connect a data cable to the brick's PC socket, then select EV3 in WebHID. Standard firmware is retained. School device policy may restrict access.
Transfer and verify	Upload, read back and compare all bytes. Sends a short completion tone after verification. Does not start the program.
Transfer and run	Upload and verify, then send a start command. Motors may move immediately.
Stop program	Send commands to stop the user program and all motors. Requires a connection. The brick's Back button also stops execution.
Disconnect	End browser communication. This is not a command to stop the running program.
Save executable (.rbf)	Download an executable to the computer. This alone does not transfer it to EV3.
Connected devices / values	Read sensor names, modes and values on inputs 1-4, and motor types and angles on outputs A-D. Auto-update reads about every 0.5 s and pauses during transfer.

Use Chrome on a supported desktop or Chromebook. Close other EV3 apps or connected tabs if a port is busy. If Bluetooth fails, disconnect and try USB. Managed devices may require Web Serial / WebHID permission from the school administrator.

Enter valid names, check for generation errors, transfer and verify, disconnect, then select the folder and program on the brick to run independently. Transferring to the same folder and program overwrites that file. Stop execution before replacing it.

3. Start and parallel execution

Start runs the connected commands from top to bottom. Detached experiments are saved in the editable project but are not included in the executable unless reachable from a start, event or called definition. An unused My Block is not executed simply because it is on screen.

Multiple Start blocks run tasks in parallel on the brick. For example, one task can perform a timed motor move while another reads an ultrasonic sensor and records CSV samples. The browser does not have to remain connected.

Ordinary numeric variables, text variables, lists, movement settings, wheel diameter, track width, ramp settings, stop behavior, PID state and timer are shared. Parameters and local working areas of My Block calls are independent for each task. Do not depend on one Start initializing another before it runs. Use explicit sequencing or a message to coordinate.

Each motor should have one responsible task. Later commands to the same port change its operation. Sensor tasks sharing a port must use compatible modes. Conflicting color/reflected-light or infrared modes across parallel tasks are rejected at generation.

With Start tasks only, all motors stop when all tasks finish, not when just one finishes. Sensor events and message receivers keep their watchers alive. End program stops every task.

4. Events and messages

Block	Meaning
When a condition becomes true	Run when false changes to true. A true condition at program start also triggers once. It does not repeatedly trigger while true.
When a number changes	Take the initial value as a baseline and trigger on subsequent changes. Noise can cause changes; use a threshold condition when appropriate.
When I receive a message	Start this receiver for a message with the matching name. Multiple receivers can share a name.
Broadcast	Trigger receivers and immediately continue.
Broadcast and wait	Continue after all matching receivers finish. With no receiver, continue immediately.

Watchers wait about 20 ms after reading and processing; this is not an exact sampling period. The same event does not overlap itself. Brief changes while its handler is busy may be missed. A condition must become false again before a new true edge triggers it.

Message names are 1-40 characters, case-sensitive, with up to 32 different names. Messages coordinate tasks inside one EV3 program; they are not wireless messages between bricks. Busy receivers coalesce additional messages into one later run rather than queuing every message individually. Cycles in broadcast-and-wait dependencies are rejected. A receiver containing forever never completes for a waiting sender.

Example: Start -> broadcast Measure and wait -> green light. Receiver Measure -> start CSV Message -> repeat 10 times [record distance on input 2; wait 0.1 s] -> close CSV. Assign CSV opening and closing to one task.

5. Motors

L / M, single / pair

L and M motors each have separate single-motor and two-motor blocks. Single blocks select A-D. Pair blocks select A+B, A+C, A+D, B+C, B+D or C+D. Use matching motor types for a pair. These blocks retain their own ports and do not use Movement's shared left/right setting, so an arm can operate independently.

Block family	Inputs and behavior
Run continuously	Speed -100 to 100%. Negative reverses direction; 0 stops. Returns immediately.
Run for degrees	Nonzero speed; angle 1-36000 motor-shaft degrees. Waits for completion.
Run for rotations	Same behavior after converting rotations to degrees. 1 rotation = 360 degrees; converted angle must be 1-36000 degrees.
Run for seconds	Nonzero speed; duration greater than 0 and at most 60 seconds. Decimal seconds are allowed. Waits for completion.
Set motor speed	Update regulated speed. Intended for motors already being controlled.
Run at power	Direct power -100 to 100%, without regulated speed feedback. Load changes actual speed.
Actual speed reporter	Read the selected motor's measured speed as a percentage.
Set end behavior	Set brake, coast or hold behavior for subsequent finite motor moves.
Stop motor using mode	Select one port, a pair or all ports, and explicitly choose brake, coast or hold. Other motors continue.
Stop all motors	Send the legacy standard-firmware all-motor stop command; execution continues.

Continuous commands need a wait, condition or continuing task afterward. If the whole program ends immediately, motors stop immediately. Shaft angles are not the same as arm or wheel angles after gearing. Speeds and angles are passed as integers, truncating fractions toward zero. Negative direction is specified with speed while the target rotation remains positive.

Stopping and paired angle control

Coast releases the drive. Brake applies braking briefly (about 100 ms) then releases. Hold maintains control while the program is active; it does not promise permanent holding after program termination. Set end behavior applies where supported by the current finite-control implementation. Explicit stop overrides the stored setting. Actual stopping depends on load and hardware.

Teacher settings offer target-angle priority, experimental stop-timing correction and EV3 synchronized control. Finishing at the same target angle does not automatically mean stopping at the same instant. Timing correction adjusts speed using encoder progress on the brick, and stops with a low tone if progress stalls too long. The record samples completion signals every 20 ms; it is not an exact physical-stop measurement. Continuous pair commands do not use angle-based correction.

Acceleration and deceleration

Set ramp profile selects none, trapezoid or S-curve and acceleration / deceleration durations in seconds. A trapezoid changes speed linearly; S-curve smooths the transitions. These are experimental software profiles, not an industrial motion controller. Short moves may not reach the requested cruising speed. Use low speeds and compare actual angle and timing. The shared setting also applies to finite steering and separate-speed angle/time moves. Continuous commands, power commands and immediate stops are excluded. Verify hardware behavior.

6. Movement

Movement ramp profiles

The same ramp setting is available in Motors and Movement, below wheel diameter / track width. The last executed setting is shared, including changes inside a My Block. Simply placing a setting does not activate it. Do not change the same motors or settings concurrently from multiple tasks.

Movement command	Ramp behavior
Forward / backward distance	Supported; needs movement motors and wheel diameter.
Spin / pivot turns	Supported; also needs track width.
Finite steering angle / time	Supported; angle means the outer motor shaft.
Finite separate-speed angle / time	Supported; angle means the faster motor. Signs and requested speed ratio are retained in target calculations.
Continuous steering / separate speeds	Excluded; starts immediately.
Stop moving	Excluded; brakes immediately without a deceleration delay.

The Movement ramp profiles sample uses left B / right C: steering 25, speed 30%, three seconds, with one-second acceleration/deceleration, first trapezoid then S-curve. It finishes with separate speeds 30% / 15% for 360 degrees of the faster motor. Check actual wiring and lift wheels for initial tests.

Finite angles prioritize each shaft's target position. Short timed moves reduce peak speed and compress ramp phases. Speed commands use integer percentages with a minimum of 1% on a moving side, so low-speed ratios are approximate. Per-motor angle targets round to integer degrees; a side rounded to zero is not started. Updates are about every 20 ms; exact stopping times and real paths are not guaranteed. Per-port stop behavior is retained and unrelated ports are unaffected. Hardware validation remains pending.

Shared motor setup

Put set movement motors left B right C before the first movement. Left and right are ordered: B/C differs from C/B. Equal ports are invalid. The setup block does not itself start motors. It takes effect when executed and is shared across the program and My Blocks. Each new run begins unconfigured; there is no implicit B+C default.

Re-executing the same setup avoids unnecessary stops. Changing to another ordered pair stops the previous pair before using the new one. Settings are not automatically restored after returning from a My Block. Avoid simultaneous commands or configuration changes from different tasks.

Generation checks reachable execution order, branches and My Block calls. A detached setup, a setup after a move, or setup on only one branch does not guarantee configuration. Parallel-task order cannot be assumed; the executable also checks configuration at runtime and stops with explicit error 23 if missing. An older project needs a new setup and wiring review.

Setting or action	Inputs and meaning
Wheel diameter	10-200 mm, resolution 0.1 mm. Measure the outside diameter. Required before distance moves and turns.
Track width	10-1000 mm, resolution 0.1 mm. Distance between wheel centers. Required for spin and pivot turns.
Steering: continuous / degrees / seconds	Steering -100 to 100; speed -100 to 100%. Finite moves require nonzero speed. Angle 1-36000 degrees of the outer motor; time greater than 0 and at most 60 s.
Separate left/right speeds: continuous	Each speed -100 to 100%. A zero side brakes. Starts immediately and continues until another command or program termination.

Setting or action	Inputs and meaning
Separate left/right speeds: degrees / seconds	Angle refers to the faster motor; time uses seconds. Choose nonzero movement and the stated finite angle / time ranges.
Spin right / left	Robot angle 1-3600 degrees, nonzero speed -100 to 100%. Wheels rotate in opposite directions.
Pivot right / left	Same robot-angle range. The inner wheel brakes; only the outer wheel moves.
Forward / backward	Distance 1-1000 cm and nonzero speed -100 to 100%. Wait for completion. Negative speed reverses the named direction.
Stop moving	Brake only the shared movement pair and continue. Requires motor setup but no diameter or track width.

For positive forward speed, steering 0 is straight, +25 curves right, +50 pivots right, +100 spins right; negative steering mirrors these to the left. Separate speeds 30/15 curve right, 15/30 curve left, 30/-30 spin right and 0/30 pivot left. Wheel mounting and ordered ports must match the robot.

Geometry

Distance motor angle = distance_cm x 10 / (pi x diameter_mm) x 360.

Spin motor angle = track_width / wheel_diameter x robot_turn_angle. Pivot motor angle is twice the spin value. With diameter 56 mm and track 112 mm, a 90-degree right spin uses left +180 / right -180 motor degrees; a right pivot uses left +360 while the right wheel stays stopped.

These formulas assume direct drive, suitable L motors and no slip. Gear ratios, tire deformation and floor friction affect the real turn. Set the actual dimensions and test. A setup My Block can set diameter and track; a movement My Block can inherit the caller's shared ports without port fields of its own.

7. Sensors

Input sensors use ports 1-4. Motor encoders and actual speed use output ports A-D. Battery and timer refer to the whole brick/program, not an input port. Round reporters fit numeric inputs; comparison blocks fit condition inputs.

Distance, color and touch

Sensor block	Value or condition
Ultrasonic distance / comparison	0-255 cm. Program readings truncate the fractional part; the browser monitor may show decimals. Compare using <, =, >, <=, >= or !=.
Color detected / color number	0 none, 1 black, 2 blue, 3 green, 4 yellow, 5 red, 6 white, 7 brown.
Reflected light / comparison	0-100%. Use the measured black/white levels to choose a threshold. Dedicated calibration and ambient-light blocks are not included.
Touch pressed / released	Current switch condition.
Touch value	1 pressed, 0 released.

Color recognition and reflected light are different sensor modes. Avoid reading both from parallel tasks. Likewise, infrared proximity, remote and beacon are different modes. A mode conflict can prevent generation. Reading is performed on the brick at execution time, not captured once in the browser.

Encoders, timer and battery

Block	Meaning
Motor rotation angle	Signed cumulative shaft angle; does not wrap to zero at 360 degrees.
Reset motor angle	Set the selected encoder count to zero; does not physically home the mechanism.
Motor actual speed	Measured motor speed percentage.
Timer	Seconds since program start or the last reset; decimal values supported.
Reset timer	Reset the shared timer to zero.
Battery voltage	Whole-battery voltage in V, not remaining-charge percentage.
Battery current	Whole-brick consumption in A including connected devices; not an individual motor's current.

Brick-button blocks are retained internally for old projects but hidden from the new-block palette. They support up, down, left, right, center or any button, as a condition or a 0/1 value. The Back button remains the normal stop operation.

Gyro

Angle returns cumulative signed degrees; angular speed returns signed degrees per second. The comparison block selects angle or angular speed and a comparison operator. Reset angle stores the current value as this program's zero; it does not recalibrate the physical sensor. Keep the robot still during reset. The browser monitor shows raw values, without that program offset.

Test first without motors: show and log angle and angular speed, rotate about 90 degrees, return, turn in the opposite direction, exceed 360 degrees and reset. Check drift while stationary and sensor mounting direction. Hardware validation is still required.

Infrared

Proximity is relative 0-100, with smaller values nearer; it is not centimeters. Remote and beacon use channels 1-4. Remote comparison checks the selected combination exactly. Beacon detected should be checked before using heading or proximity. Heading is relative -25 to 25 (left negative), not degrees; beacon proximity is relative 0-100.

Code	Remote buttons	Code	Remote buttons
0	None	6	Upper left + lower right
1	Upper left	7	Lower left + upper right
2	Lower left	8	Lower left + lower right
3	Upper right	9	Beacon
4	Lower right	10	Upper left + lower left
5	Upper left + upper right	11	Upper right + lower right

Browser monitoring

Read values shows detected device, current mode and current value. Auto-update reads sequentially about every 0.5 s and pauses for transfer. It does not switch sensor modes. The IR remote/beacon monitor shows the first CH1 value. A program must explicitly read a mode for that mode's values. Browser monitoring and program measurements are separate operations.

8. Sound

Play tone accepts frequency 250-10000 Hz, duration 1-30000 milliseconds and volume 0-100. It waits for the tone to finish. The duration unit is ms, unlike the wait and timed-motor blocks that use seconds. 1000 ms = 1 s.

Use Sound only to verify transfer and independent execution. User audio files and speech playback are not implemented. The short transfer-completion tone is a separate notification, not proof that the transferred program was executed.

9. Control

Block	Behavior
Wait seconds	0-60 seconds, including decimals; new palette default 1 second.
If	Test once and run the enclosed commands only if true.
If / else	Run the upper branch if true, otherwise the lower branch. The else label sits between the two statement areas.
Repeat	0-1000 iterations; new default 10. Zero skips the body.
Forever	Repeat without a fixed end. Use program stop or End program to finish.
Wait until	Continue when the condition becomes true. Reads conditions repeatedly.
Repeat until / while	Repeat according to the selected condition behavior. Ensure the condition can change.
End program	Stop all tasks and motors; does not return to a caller.
Exit this My Block	Return to the caller without automatically stopping a motor. Only valid in a command My Block.
Return value(s) and exit	Finish a numeric-result My Block immediately; provide all required results.

Stopping a motor is different from ending the program. A motor stop allows the next command, such as a sound or measurement. End program terminates the whole program, including parallel tasks. Legacy millisecond waits remain readable in older projects.

For sensor-controlled motion: run a motor continuously -> wait until distance < 15 -> stop the motor. For line following: forever [if reflected light < threshold, set left/right speeds 10/30; else 30/10; wait a short interval]. This repeatedly reads the sensor while the robot moves.

10. Operators and PID

Numeric, comparison and logic

Add, subtract, multiply and divide accept numeric reporters, variables, parameters or sensor values. Division by zero is invalid. Comparisons include <, <=, =, !=, >= and >. And requires both conditions, or requires either, and not reverses a condition. Boolean true/false are constants.

Arithmetic, variables and parameters retain decimals using EV3 32-bit floating point (about seven significant digits). Values such as 0.1 have rounding error. Prefer a tolerance or >= / <= when checking whether a measured target was reached. Motor speed, angles and iteration counts truncate fractions toward zero; durations convert decimal seconds to milliseconds.

Function	Meaning and domain
Absolute value	Magnitude without sign. abs(-7.25) = 7.25.
Floor / ceiling	Nearest integer below / above; floor(-2.3) = -3.
Square root	Input must be nonnegative.
sin / cos / tan	Input in degrees. tan at 90 + 180n degrees is invalid.
asin / acos / atan	Result in degrees; asin/acos inputs must be between -1 and 1.
ln / log	Natural / base-10 logarithm, input strictly positive.
e^ / 10^	Exponential, within finite single-precision limits.
Modulo	Remainder with the divisor's sign; -7 mod 3 = 2. Divisor must be nonzero.
Random integer	Inclusive bounds -32767 to 32767; fractional bounds truncate toward zero. Lower <= upper. Chosen on EV3 at runtime.
Round	floor(value + 0.5); round(-1.5) = -1.
Minimum / maximum	Smaller / larger of two numbers.
Clamp	Limit a number to inclusive lower / upper bounds. Lower <= upper.

Invalid domains or non-finite results cause an error. Some can be detected before transfer; values depending on sensors are checked at runtime. Clamp calculated continuous speeds to -100..100. Finite movement cannot use zero speed, so handle zero separately.

PID controller

PID takes controller ID, target, measured input and gains kp, ki, kd, and returns a numeric output. Error is target minus measured. P acts on current error, I on accumulated error over elapsed time, D on error change over elapsed time. Gains multiply those terms; they are not measured P/I/D input values.

Use a separate ID for each independent controller. PID state is shared by ID and reset at a new run; Reset PID clears the selected controller's history. Call it before a new control experiment. PID does not directly drive a motor or automatically limit output. Put the output into Clamp and then a suitable speed command. Choose a repeat interval and tune kp first, then ki and kd carefully on the real mechanism. The first call after reset uses only P. Calls within the same millisecond reuse the previous output. Call timing affects I and D. Different tasks should not update the same controller concurrently.

The no-motor PID sample uses target 10 and measured 4, displays results and records CSV PID. Test calculations before using output to move hardware. This is experimental software control, not a certified motion controller.

11. Display

Display commands draw on the EV3's monochrome 178 x 128 pixel screen. The upper-left corner is x=0, y=0; x grows rightward and y downward. Drawing returns immediately; add a wait to observe a fixed display or a loop to refresh sensor readings.

Block	Inputs
Clear screen	Make the screen white.
Display text	Fixed text, 1-120 characters, including Japanese. Small/medium/large text is rasterized at 12/16/24 pixels. Text wraps within the screen width.
Display image	Smile, sad face, surprised face, heart, check, cross or up/down/left/right arrows. 48 or 96 pixel square.
Display number	Numeric reporter, variable, operator or sensor. 0-3 decimal places, up to the first 10 characters including sign and decimal point.

Coordinates must be inside x=0..177, y=0..127 and the whole drawing must fit. Numeric fields occupy 80 x 9 pixels for small or 160 x 19 for large. A command clears its own drawing area; other areas remain. Clear the screen first if older, longer text would remain visible.

Fixed text is converted to a bitmap in the browser and embedded in the executable, so no extra image file is needed on the brick. Glyphs depend on the author's fonts. Dynamic text-variable display and importing arbitrary image files are not supported. More graphics increase the executable size. Parallel drawings are serialized one block at a time; later drawings overwrite the same area.

12. Light

Set brick light selects green, red or orange and steady, blink or slow blink behavior. Turn brick light off switches it off. These commands do not wait and work on the brick when disconnected. Add waits to observe a sequence. At program termination the normal firmware light behavior resumes.

Example: green steady -> wait 1 s -> red steady -> wait 1 s -> orange blink -> wait 1 s -> green slow blink -> wait 1 s -> off. These colors are the brick indicator, separate from color-sensor readings and editor category colors.

13. Data: text and numeric lists

Text

Block family	Behavior
Text constant / join	Enter text or concatenate two texts.
Length / letter	Count Unicode characters; combining characters count separately. Letter positions start at 1, with index 1-120. Beyond the text's length returns empty text.
Contains / equals	Test substring presence or whole-text equality. Empty text is contained.
Number as text	Convert with about seven significant digits.
Text as number	Parse ASCII signs, digits, decimal point and exponent without spaces. Invalid text causes an error.
Set / get text variable	Named text storage separate from ordinary numeric variables.

Text variable names are 1-40 characters, with at most 16 variables. Values begin empty each run. Text capacity is 120 UTF-8 bytes, including after joining (typically up to 40 Japanese characters). Overflow is an error. Text does not fit numeric speed inputs; convert if appropriate. My Block parameters and results currently accept numbers, not text.

Numeric lists

Clear, add, replace item, delete item, item reporter, length, contains, sum, mean, minimum and maximum operate on a named numeric list. Lists support decimals and are shared throughout the program. They are not text lists.

List names are 1-40 characters; at most 16 lists, each with 1000 items. Indices start at 1. Reading, replacing or deleting an absent item is an error. Sum of an empty list is 0; mean/minimum/maximum require at least one item. Lists begin empty each run; record CSV if values must survive.

Example: clear Measurements -> repeat 10 [add reflected light to Measurements; wait 0.1 s] -> record mean of Measurements to CSV. Use exactly the same name in each list block.

14. Recording: CSV logs

Start CSV log creates Log.csv in the current project folder and resets its elapsed time and row count. An existing file with the same name is overwritten. Record item/value writes elapsed seconds, an item label and a numeric or text value. Close CSV finishes the file. Record only between start and close.

Logging runs on the brick and does not require a computer connection. Up to 5000 data rows are allowed per start. Add an explicit wait inside sampling loops to choose the interval; actual timing includes sensor and command execution. One shared log is active, so coordinate tasks and assign opening/closing to one task.

After execution stops, reconnect, open the connection panel, enter the same project folder and log name without extension, then choose Export CSV from EV3. The downloaded file includes UTF-8 support for spreadsheet use. Exporting does not delete the log on the brick. A different project folder cannot retrieve the file merely by reusing the log name.

Example: start Distance -> repeat 20 [record item ultrasonic, value distance input 2; wait 0.1 s] -> close CSV -> stop program -> export Distance.csv. Browser communication logs are separate diagnostic files.

15. Variables

Create variable opens a naming dialog. Numeric variable reporters appear individually by name in Variables; set and change blocks choose a name from a dropdown. No extra “variable” prefix is shown on the reporter.

Set replaces the value. Change adds a number, including a negative number or decimal. Variables begin at 0 on each execution and are shared by all tasks and My Blocks. Names are 1-40 characters and must not conflict with parameter names. Up to 32 ordinary numeric variables are supported.

For values that must survive a run, record CSV. Autosaving the editable project does not save the EV3 program's current variable values. Parallel tasks sharing a variable should coordinate their changes explicitly.

16. My Blocks and results

Choose Create My Block, enter a name, add numeric parameters if needed, and choose command-only or numeric results. Up to 32 definitions are allowed; names are 1-60 characters. Each definition can have up to 8 parameters and up to 8 named numeric results. Put commands in the definition, then take a matching call from My Blocks. Defining a block does not execute it.

Use Edit My Block to change the definition through the dialog rather than a gear. Calls arrange parameter inputs horizontally. Put ordinary numbers, variables, operators or sensors in matching input shapes.

Command and numeric calls

A command call joins the statement stack and returns to the next command. A round numeric call fits a number input. It executes the definition's commands and computes the requested result. A definition with multiple results shows a result selector on the round call.

Example: Average with parameters a and b returns (a+b)/2. Use Average(440,880) in the tone frequency to play 660 Hz. SumDifference with parameters a and b and results sum / difference can return 660 or 220 for inputs 440 and 220.

Each round call runs the definition again, even when only the selected result differs. It does not reuse a previous call's measurements. Two separate calls selecting sum and difference execute the definition twice, including its sensor reads, motor commands and variable changes. To use one measurement snapshot, store inputs in variables first.

Exit this My Block returns early from a command-only definition. Return value(s) and exit returns early from a numeric definition; the result count and names must match. A single-result return cannot omit the other results in a multi-result definition. Returns do not themselves stop motors.

Changing result count or command/numeric shape requires deleting existing calls first, then recreating them after the edit. Disconnect calculation blocks before removing their result input. Renaming results updates labels; verify saved calls after edits. Recursive calls are not supported.

Parameters and call work areas are independent per task. Ordinary variables and movement/PID settings remain shared, including changes made inside a definition. Export / Import My Blocks carries definitions and required dependencies to another project. It does not silently overwrite same-name definitions. Current projects and libraries preserve numeric results and selected result names.

17. Parameters

Parameters receive numeric values supplied by a My Block call. The Parameters group is separate from Variables and displays only each parameter's name. Use a reporter inside a definition that declares that name. It cannot be assigned using Set variable or used freely outside the matching definition.

If multiple definitions use the name distance, the palette displays it once and it can be used in either matching definition. The values are still independent for each call; this is shared naming, not shared storage. A call can pass 20 to one definition and 50 to another.

To make a reusable drive block: set movement ports and diameter in the caller, then call Drive(distance,speed). Inside the definition, put the distance and speed parameter reporters into the port-free forward block. The caller's movement configuration is inherited.

18. Samples and hardware testing

Sample	Wiring / expected test
Sound only	No motors; transfer, verify, select on brick and hear the tone.
Movement ramp profiles	Left B / right C; compare ramped steering time and separate-speed angles.
Forward with shared setup	Left B / right C, diameter 56 mm, forward 20 cm at 20%; change for the real robot.
Spin / pivot turns	Set actual diameter and track width; compare left and right turns.
Stop at distance	Pair B+C and ultrasonic input 4; stop below 15 cm.
Tone on red	Color sensor input 3; sound changes when red is detected.
Compare stop modes	Motor B; compare coast, brake and hold.
Separate speeds for angle / time	Matching B+C; 30/15% for 180 degrees of faster side, then opposite directions for 0.5 s.
Drive and measure in parallel	B+C and ultrasonic input 2; timed driving and CSV Parallel.
Stop on touch	B+C, touch input 1; condition event ends the program.
Measure using messages	Ultrasonic input 2; waiting sender and CSV Message.
Battery / PID	No motors; display and record Battery or PID.
Gyro angle / angular speed	Gyro input 1; display and log Gyro while turning by hand.
My Block results	Average 440/880 gives 660 Hz; sum/difference sample displays 660 and 220.

Confirm actual wiring before every motor test and lift the wheels for the first run. Test positive and negative speeds, both turn directions, all intended port pairs, My Block reuse, transfer-only versus transfer-and-run, disconnected execution and execution after restarting EV3.

For shared movement setup, verify that repeating the same setup does not interrupt continuous motion, changing pairs stops the previous pair, and a new run does not inherit old settings. Test missing setup through events as well as single Start sequences.

Report application version, operating system, browser, EV3 firmware, connection type, ports, device types, expected and actual behavior and reproduction steps. Attach the .ev3lab project, exported communication log, relevant CSV and brick error photo if available. Automated and simulated tests do not establish hardware behavior.

19. Errors and limits

Generation errors shown below the workspace are not necessarily connection problems. Check them first if transfer controls remain disabled. Required configuration must execute before movement; placing a disconnected setup block is insufficient. Fill empty numeric and condition sockets with correctly shaped blocks.

If transfer succeeds but nothing moves, run the newly named file on the brick, check ports and check whether the program immediately reaches its end. A successful byte comparison establishes file transfer, not mechanical performance. For VM PROGRAM VALIDATION or another brick error, save the project and logs and report the exact message.

Resource	Limit
Project blocks / executable	1000 blocks / 64 KB executable
Ordinary numeric variables / My Blocks	32 / 32
Parameters / results per definition	8 / 8
Start, condition-event and message-receiver hats	16 combined
Message names	32, each 1-40 characters
Text variables / numeric lists	16 / 16
Text value / list items	120 UTF-8 bytes / 1000 items per list
CSV recording	5000 rows per start

Complex nesting and generated control code can reach other validation or executable-size limits before a simple resource count. Split very large programs. Invalid sensor types, disconnected devices or invalid runtime values may stop all motors with a low tone. Hardware faults and load behavior require real testing.

Not included: grade-specific language/blocks, “move N squares”, dedicated reflected-light calibration, ambient-light reporter, audio-file playback, string parameters/results, text lists, dynamic text-variable screen display or arbitrary image import. Brick-button blocks are hidden for new projects. Old projects may retain legacy blocks and require wiring/setup review.

The language switch translates editor presentation, not personal names, sample data or external operating-system/device error messages. Browser permission dialogs use the browser/OS language. Save before switching if you need the current undo history.

20. EV3 Classroom comparison and purpose

This table summarizes the existing project comparison based on LEGO Education and Afrel documentation, alongside this version's implementation. Classroom differs by OS and version. “Implemented” does not imply identical precision, file compatibility or hardware verification. Availability in app stores is not assessed here.

Feature	EV3 Classroom	EV3 Block Lab v0.25.6
Use	Installed application; ChromeOS uses Android edition	Browser application; no app installation
L/M speed, angle, rotations, duration	Supported	Implemented, single and paired blocks
Paired motion, steering, separate speeds, turns	Supported	Implemented; shared ordered movement ports
Stop modes, encoder, actual speed	Supported	Implemented, with behavior differences
Ultrasonic, color, touch, infrared	Supported	Distance, color/reflection, touch, all three IR functions
Gyro	Supported	Angle, rate, comparison and program-local zero offset; hardware checks pending
Parallel tasks, sensor events, messages	Supported	Implemented
Conditions, loops, variables, arithmetic	Supported	Implemented, including decimals
My Blocks	Supported	Numeric parameters, up to 8 results and library export/import
Brick display	Text, numbers, images	Fixed text including Japanese, numbers, 10 preset images
Light / sound	Includes recorded sound	Three light colors and generated tones; no audio files
Brick buttons	Supported	Internal legacy support; hidden from palette
Data logging	No standard Classroom logging; older EV3 Lab has it	On-brick CSV and export
USB	OS-dependent	WebHID implemented; device testing continues
Editor language	Available languages depend on edition	Japanese / English switch and manuals

Why this project matters

The purpose is to keep school-owned EV3 robots useful with pupils' Chromebooks as older Windows PCs age. AI generates code, while people apply classroom experience to learning design, operation and real-hardware testing.

The central aim is for children to enjoy programming, think, try ideas and learn from the process. Having many features is only part of that. Future work will consider age-appropriate wording and displays; those grade-specific features are not yet implemented. Hardware test participation and feedback are welcome.

References

LEGO Education system requirements:
https://education.lego.com/en-gb/product-resources/mindstorms-ev3/downloads/system-requirements/

Afrel Classroom comparison: https://afrel.co.jp/technology-info/information-ev3/ev3-software/ev3-ev3-classroom/44135/

Afrel motor comparison: https://afrel.co.jp/technology-info/information-ev3/ev3-software/ev3-ev3-classroom/44140/

Afrel events / parallel execution:
https://afrel.co.jp/technology-info/information-ev3/ev3-software/ev3-ev3-classroom/44145/

Afrel display / sound: https://afrel.co.jp/technology-info/information-ev3/ev3-software/ev3-ev3-classroom/44143/

Application: https://ev3-block-lab-prototype.davinichi-9576.chatgpt.site/?lang=en

Source: https://github.com/davinichi/ev3-block-lab

EV3 Block Lab is an unofficial experimental application, not LEGO's official software. Its .ev3lab and .ev3blocks formats are not interchangeable with Classroom or old EV3 Lab project files.